

# Sygnały i obrazy cyfrowe

## Skalowanie obrazów

Łukasz Kwieciński 284142

28 listopada 2025

## 1 Wprowadzenie

Celem ćwiczenia było zapoznanie się z interpolacją funkcji oraz skalowaniem obrazów z wykorzystaniem konwolucji. W pierwszej części należało porównać różne jądra interpolacyjne, zbadać jakość wyników przy użyciu kryterium MSE oraz sprawdzić wpływ liczby i rozmieszczenia punktów. W drugiej części zadania zaimplementowano zmniejszanie i powiększanie obrazów na podstawie splotu i metod interpolacyjnych, a następnie oceniono jakość przetwarzania. Ćwiczenie pozwala zrozumieć zależności między interpolacją, splotem a dokładnością odwzorowania sygnałów i obrazów

## 2 Interpolacja funkcji

Funkcje do interpolacji

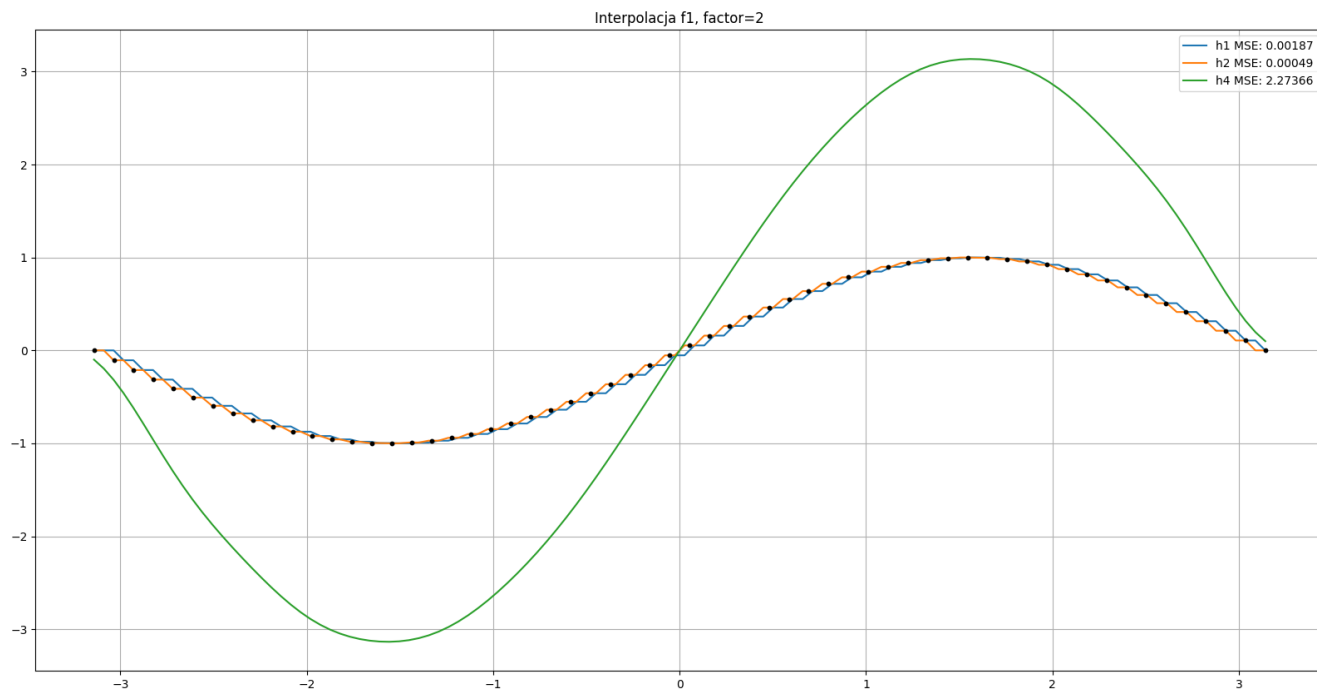
$$\begin{aligned}f_1(x) &= \sin(x) \\f_2(x) &= \sin(x^{-1}) \\f_3(x) &= \operatorname{sgn}(\sin(8x))\end{aligned}$$

Funkcje jądrowe

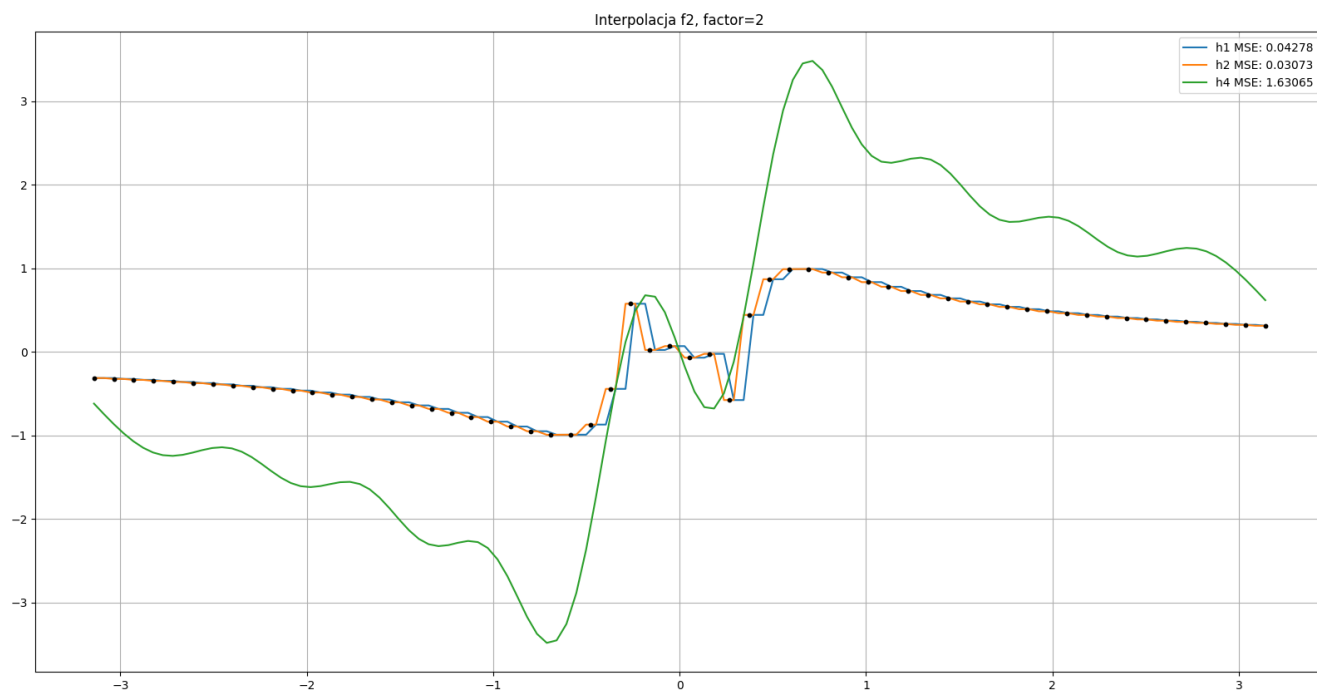
$$\begin{aligned}h_1(t) &= \begin{cases} 1 & t \in [0, 1) \\ 0 & t \notin [0, 1) \end{cases} \\h_2(t) &= \begin{cases} 1 & t \in [-\frac{1}{2}, \frac{1}{2}) \\ 0 & t \notin [-\frac{1}{2}, \frac{1}{2}) \end{cases} \\h_3(t) &= \begin{cases} 1 - |t| & t \in [-1, 1] \\ 0 & t \notin [-1, 1] \end{cases} \\h_4(t) &= \frac{\sin(x)}{x}\end{aligned}$$

## 2.1 Badanie wpływu liczby oraz rozmieszczenia punktów na jakość konwolucji

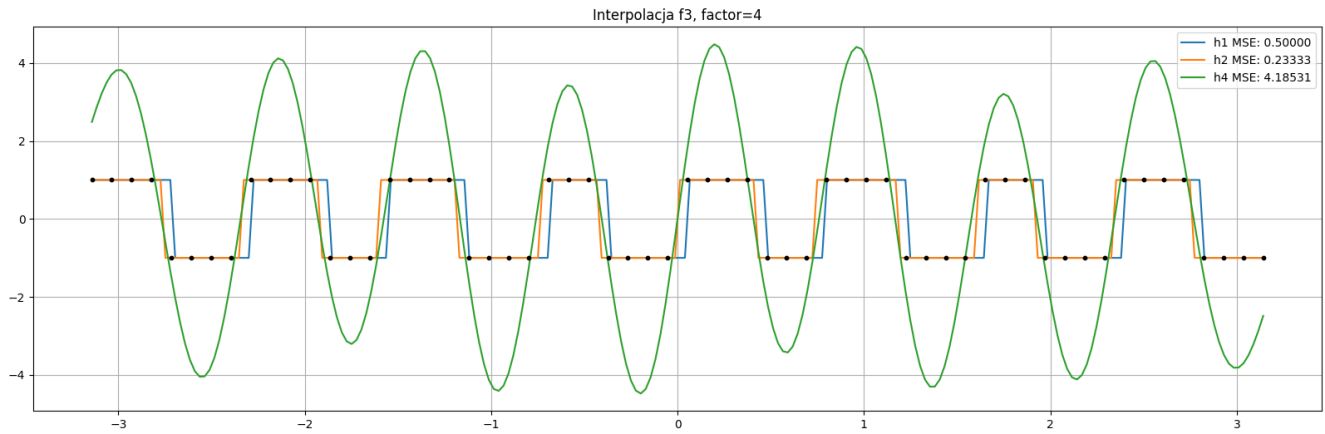
### 2.1.1 Konwolucja przy czynniku próbkowania $x_2$



Rysunek 1: Konwolucja funkcji  $f_1$  przy czynniku próbkowania  $x_2$



Rysunek 2: Konwolucja funkcji  $f_2$  przy czynniku próbkowania  $x_2$

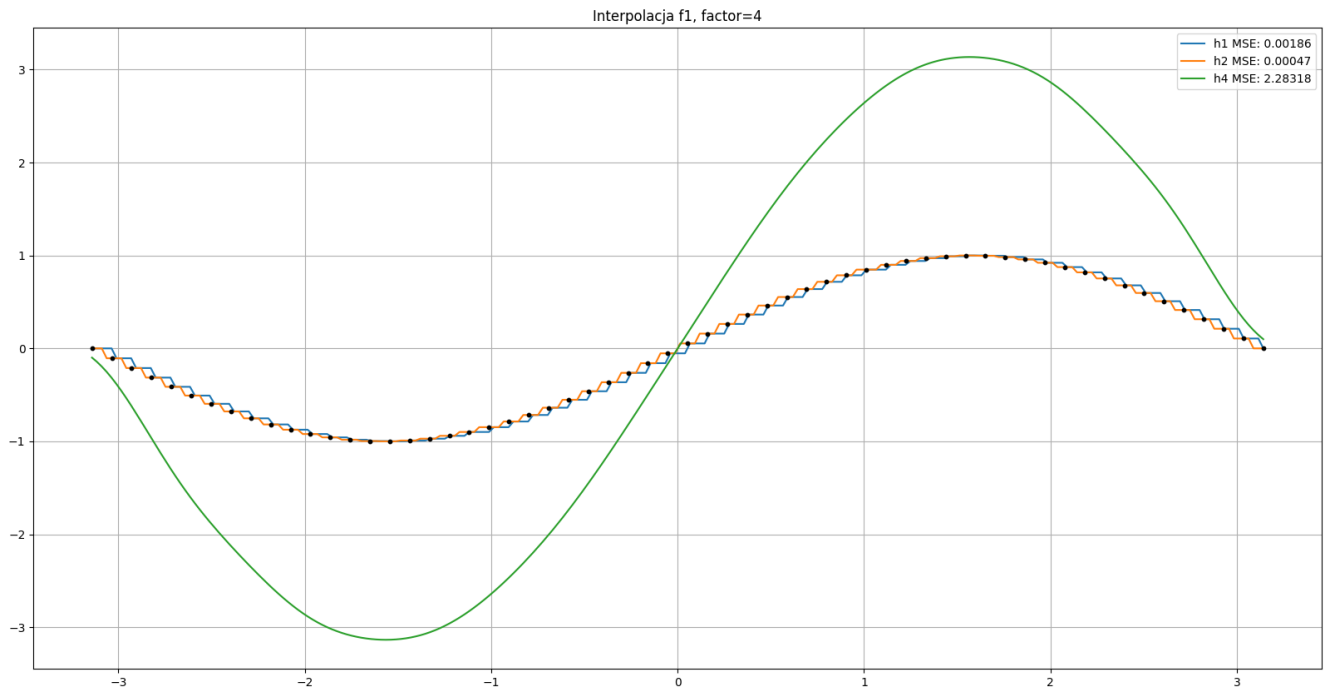


Rysunek 3: Konwolucja funkcji  $f_3$  przy czynniku próbkowania  $x_2$

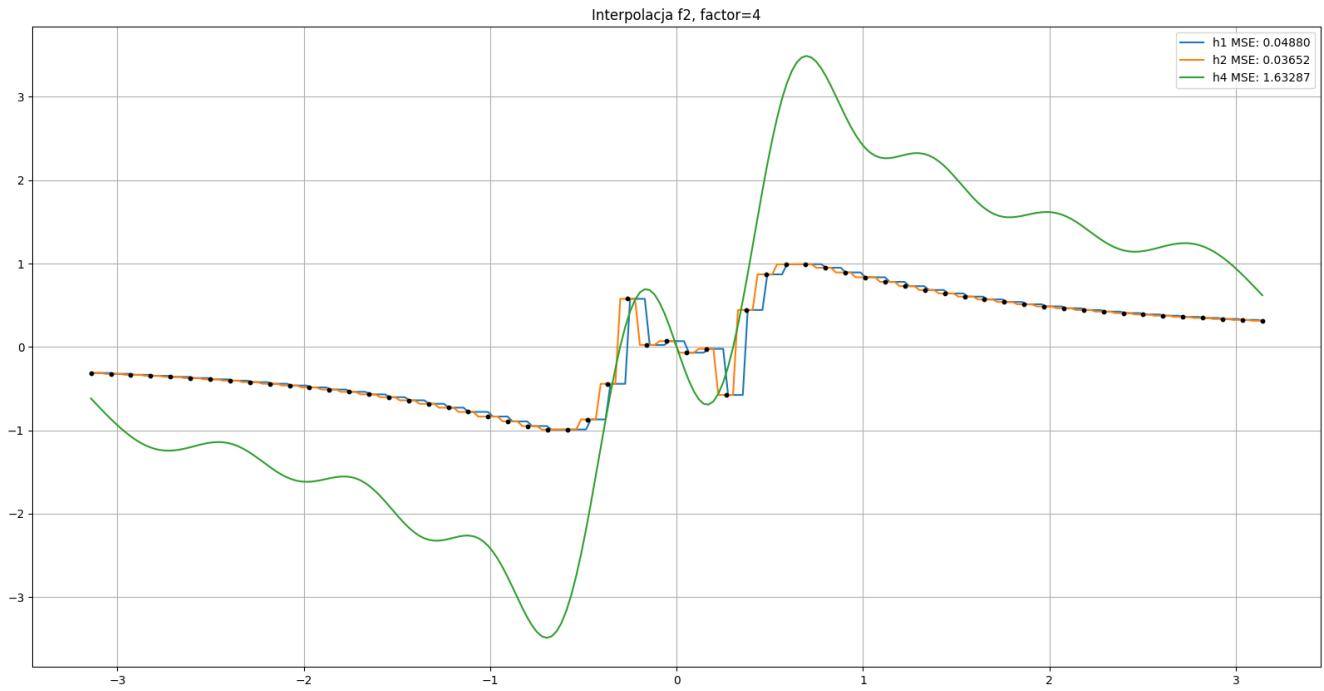
|    | f1           | f2           | f3           |
|----|--------------|--------------|--------------|
| h1 | 0.0018674464 | 0.0427758393 | 0.5000000000 |
| h2 | 0.0003919621 | 0.0307323603 | 0.2666666666 |
| h4 | 2.2736602010 | 1.6306472597 | 4.1699279715 |

Tabela 1: Wyniki MSE dla każdej z funkcji oraz odpowiadającego jądra przy czynniki próbkowania  $x_2$

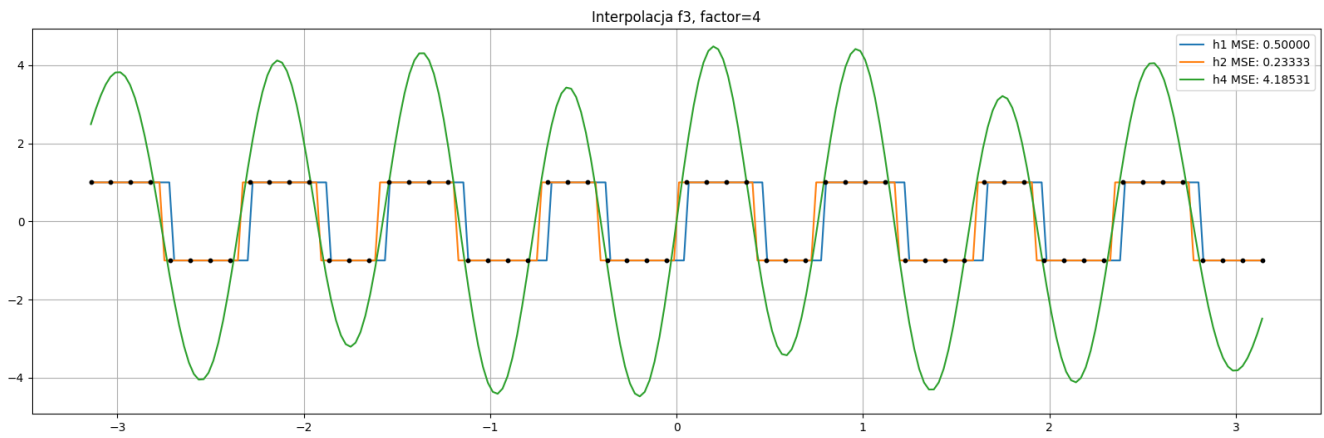
### 2.1.2 Konwolucja przy czynniku próbkowania $x_4$



Rysunek 4: Konwolucja funkcji  $f_1$  przy czynniku próbkowania  $x_4$



Rysunek 5: Konwolucja funkcji  $f_2$  przy czynniku próbkowania  $x_4$

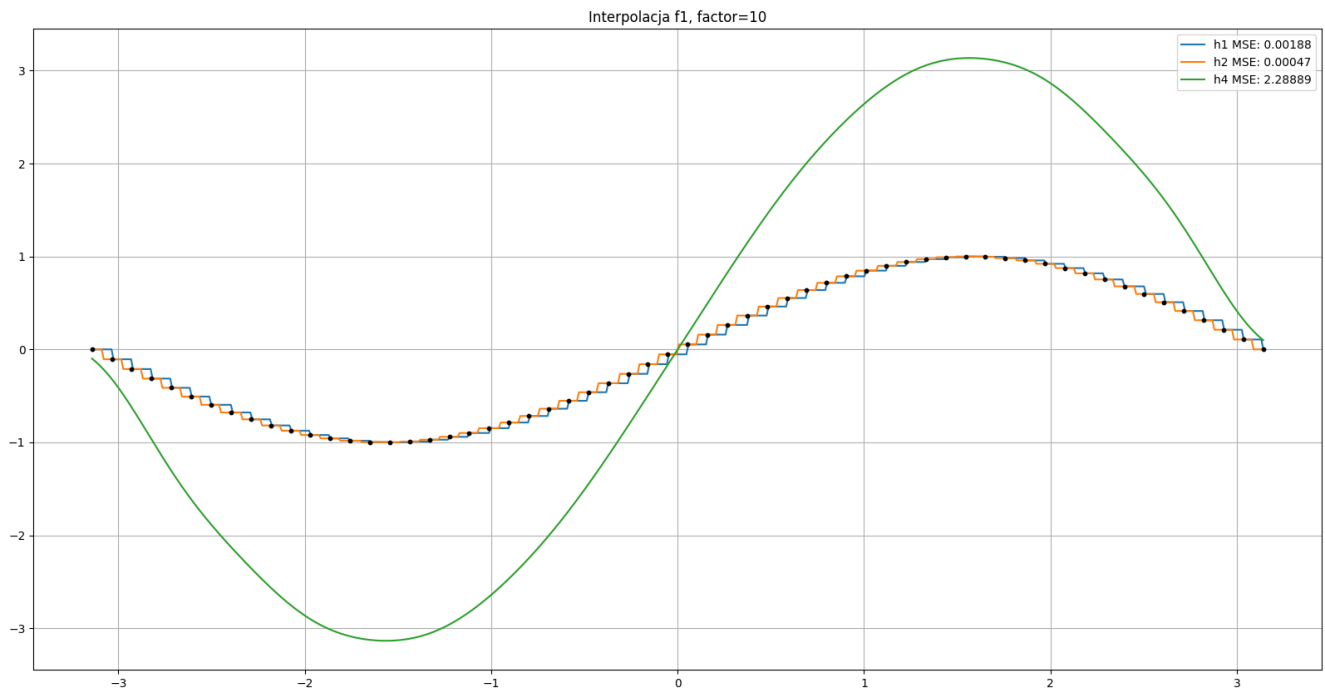


Rysunek 6: Konwolucja funkcji  $f_3$  przy czynniku próbkowania  $x_4$

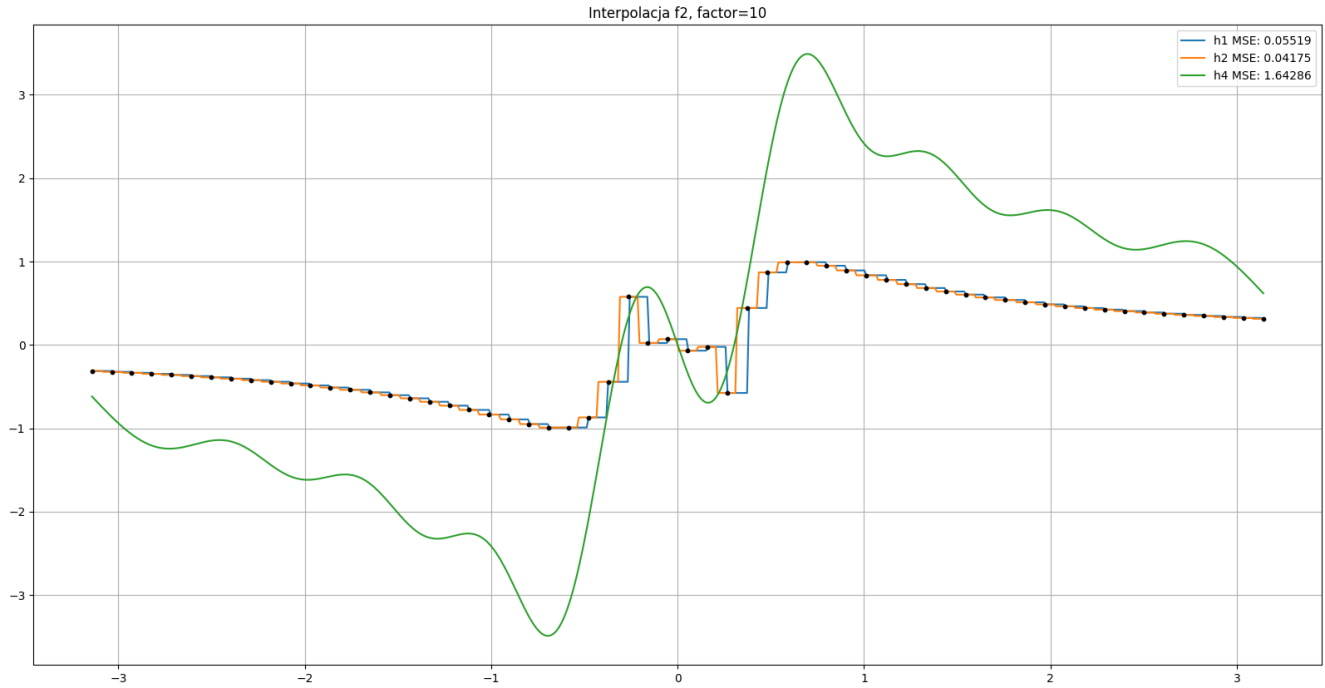
|           | <b>f1</b>    | <b>f2</b>    | <b>f3</b>    |
|-----------|--------------|--------------|--------------|
| <b>h1</b> | 0.0018570047 | 0.0487982314 | 0.5000000000 |
| <b>h2</b> | 0.0004708368 | 0.0365199140 | 0.2400000000 |
| <b>h4</b> | 2.2831783846 | 1.6328702094 | 4.1853106801 |

Tabela 2: Wyniki MSE dla każdej z funkcji oraz odpowiadającego jądra przy czynniki próbkowania  $x_4$

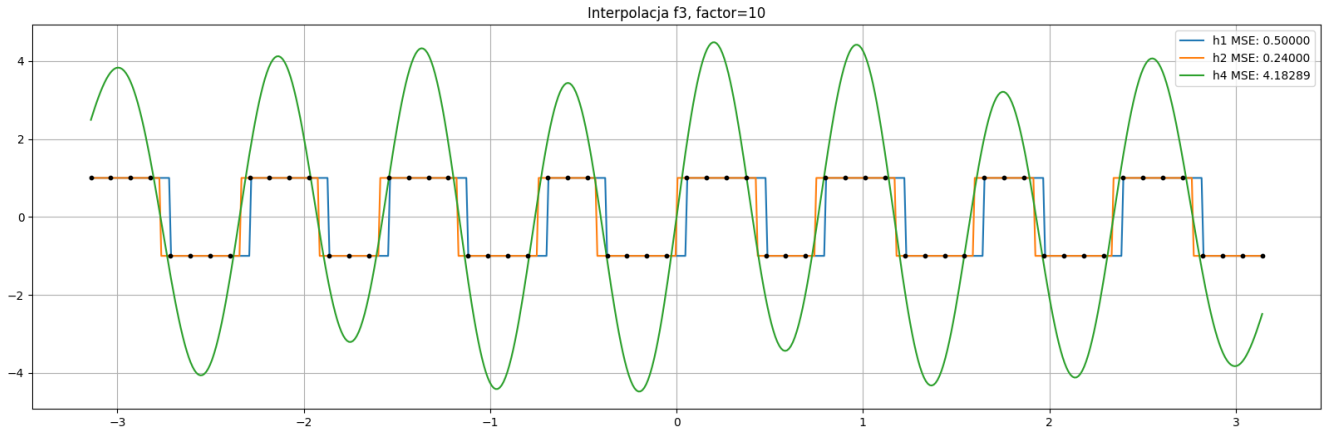
### 2.1.3 Konwolucja przy czynniku próbkowania $\times 10$



Rysunek 7: Konwolucja funkcji f1 przy czynniku próbkowania  $\times 10$



Rysunek 8: Konwolucja funkcji f2 przy czynniku próbkowania  $\times 10$



Rysunek 9: Konwolucja funkcji  $f_3$  przy czynniku próbkowania  $x_{10}$

|           | $f_1$        | $f_2$        | $f_3$        |
|-----------|--------------|--------------|--------------|
| <b>h1</b> | 0.0018764664 | 0.0551891401 | 0.5000000000 |
| <b>h2</b> | 0.0004715642 | 0.0417475609 | 0.2400000000 |
| <b>h4</b> | 2.2888872970 | 1.6428616096 | 4.1828861038 |

Tabela 3: Wyniki MSE dla każdej z funkcji oraz odpowiadającego jądra przy czynniki próbkowania  $x_{10}$

#### 2.1.4 Wnioski

##### 1. Wpływ jądra konwolucji

- Kernel prostokątny (**h1**) daje najmniejsze błędy dla funkcji  $f_2$  i  $f_3$  szczególnie przy wyższych czynnikach próbkowania.
- Kernel trójkątny (**h2**) znacząco zmniejsza MSE w porównaniu do jądra prostokątnego.
- Kernel **h4** daje bardzo wysokie MSE dla wszystkich funkcji, co sugeruje, że nie jest odpowiedni w tym zadaniu.

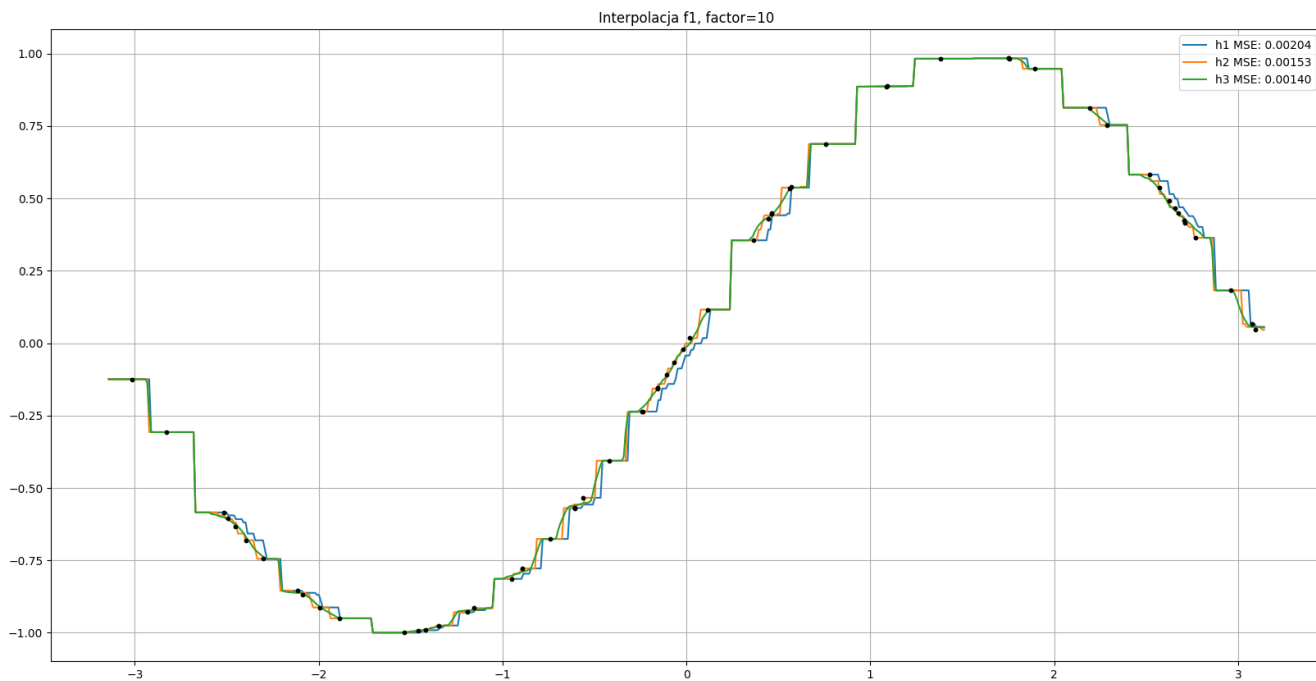
##### 2. Wpływ czynnika próbkowania

- Zwiększanie czynnika próbkowania ( $x_2$ ,  $x_4$ ,  $x_{10}$ ) nie zmienia znacząco MSE dla kernelów **h1** i **h2**, co oznacza, że jakość konwolucji zależy głównie od wyboru jądra, a nie liczby punktów w tym zakresie.
- Dla kernelu **h4** MSE rośnie nieznacznie wraz ze wzrostem czynnika próbkowania, potwierdzając jego nieoptymalność.

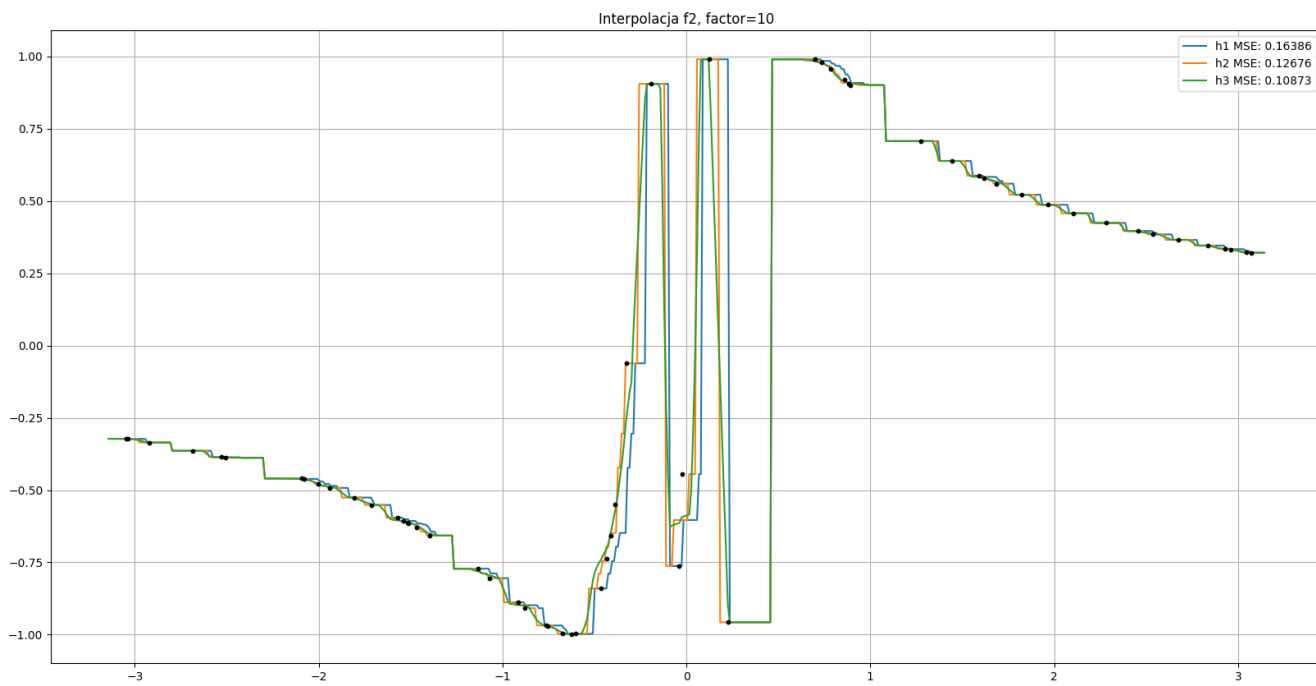
Najlepsze wyniki uzyskano stosując kernel trójkątny (**h2**). Wybór odpowiedniego jądra ma większy wpływ na jakość konwolucji niż zwiększanie liczby punktów próbkowania.

## 2.2 Badanie wpływu rozmieszczenia punktów na jakość interpolacji

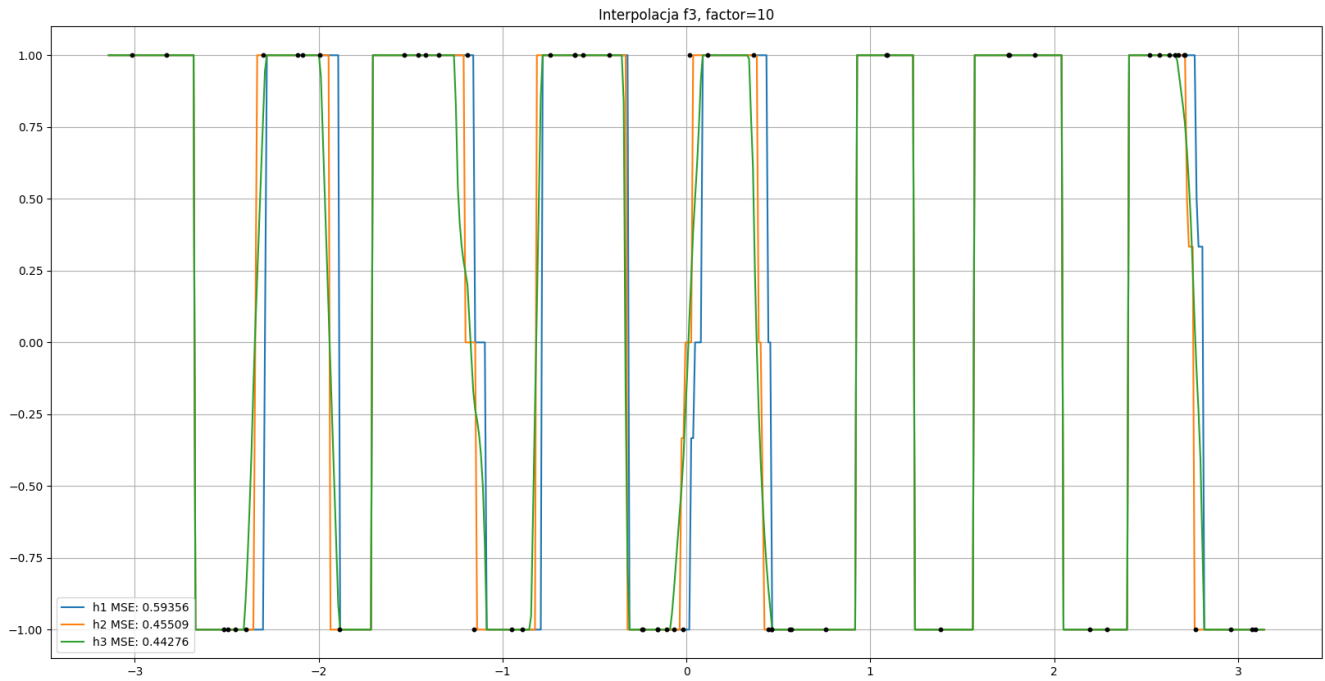
### 2.2.1 Interpolacja przez konwolucje przy losowych punktach



Rysunek 10: Konwolucja funkcji f1 z losowymi punktami



Rysunek 11: Konwolucja funkcji f2 z losowymi punktami



Rysunek 12: Konwolucja funkcji  $f_3$  z losowymi punktami

|    | f1           | f2           | f3           |
|----|--------------|--------------|--------------|
| h1 | 0.0020397498 | 0.1638629083 | 0.5935648148 |
| h2 | 0.0015314780 | 0.1267589336 | 0.4550925925 |
| h3 | 0.0014038553 | 0.1087258508 | 0.4427589839 |

Tabela 4: Wyniki MSE dla każdej z funkcji oraz odpowiadającego jądra przy losowych punktach

## 2.2.2 Wnioski

### 1. Wpływ jądra konwolucji i czynnika próbkowania:

- Kernel prostokątny ( $h_1$ ) daje największe błędy i generuje zniekształcenia sygnału.
- Kernel trójkątny ( $h_2$ ) znacząco poprawia jakość interpolacji, zapewniając gładzsze odwzorowanie funkcji.
- Zwiększanie czynnika próbkowania nie wpływa znacząco na jakość interpolacji dla kernelów  $h_1$  i  $h_2$ .

### 2. Wpływ rozmieszczenia punktów:

- Interpolacja na losowych punktach powoduje wyższe błędy niż przy równomiernym rozmieszczeniu.
- Kernel  $h_1$  daje największe błędy, natomiast  $h_2$  poprawia jakość.
- Kernel  $h_3$  daje najlepsze wyniki przy losowym rozmieszczeniu punktów.
- Funkcja  $f_1$  jest stosunkowo odporna na losowe rozmieszczenie, natomiast  $f_2$  i  $f_3$  są bardziej wrażliwe.



### 3 Skalowanie Obrazów



Rysunek 13: Oryginalne zdjęcie czarno białe.

#### 3.1 Zmniejszanie obrazów czarno-białych

##### 3.1.1 Pomniejszanie obrazu przy użyciu jądra uśredniającego i powiększanie przy użyciu w.w jąder

Jądro uśredniające:

$$K_m = \frac{1}{N} \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix}$$



Rysunek 14: Zdjęcie pomniejszone  $\times 0.5$  z użyciem jądra uśredniającego



(a) Zdjęcie pomniejszone powiększone  $\times 2$  z użyciem jądra  $h_1$



(b) Zdjęcie pomniejszone powiększone  $\times 2$  z użyciem jądra  $h_2$



(c) Zdjęcie pomniejszone powiększone  $\times 2$  z użyciem jądra  $h_3$



(d) Zdjęcie pomniejszone powiększone  $\times 2$  z użyciem jądra  $h_4$

Rysunek 15: Porównanie powiększeń  $\times 2$  z użyciem różnych jąder.

|     | h1          | h2          | h3          | h4          |
|-----|-------------|-------------|-------------|-------------|
| MSE | 668.6283517 | 487.8625973 | 506.3543381 | 583.7250058 |

Tabela 5: Wyniki MSE dla różnych jąder

### 3.1.2 Powiększanie obrazów przy użyciu *max pooling* i powiększanie przy użyciu *nearest neighbor*

Jądro *nearest neighbor*:

$$K_m = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}$$

Jądro *max pooling*:

$$K_m = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix}$$



Rysunek 16: Zdjęcie pomniejszone o x0.5 z użyciem jądra *max pooling*



Rysunek 17: Zdjęcie pomniejszone powiększone o  $\times 2$  z użyciem jądra *nearest neighbor*

Wynik MSE: 30.97465891

### 3.1.3 Wnioski

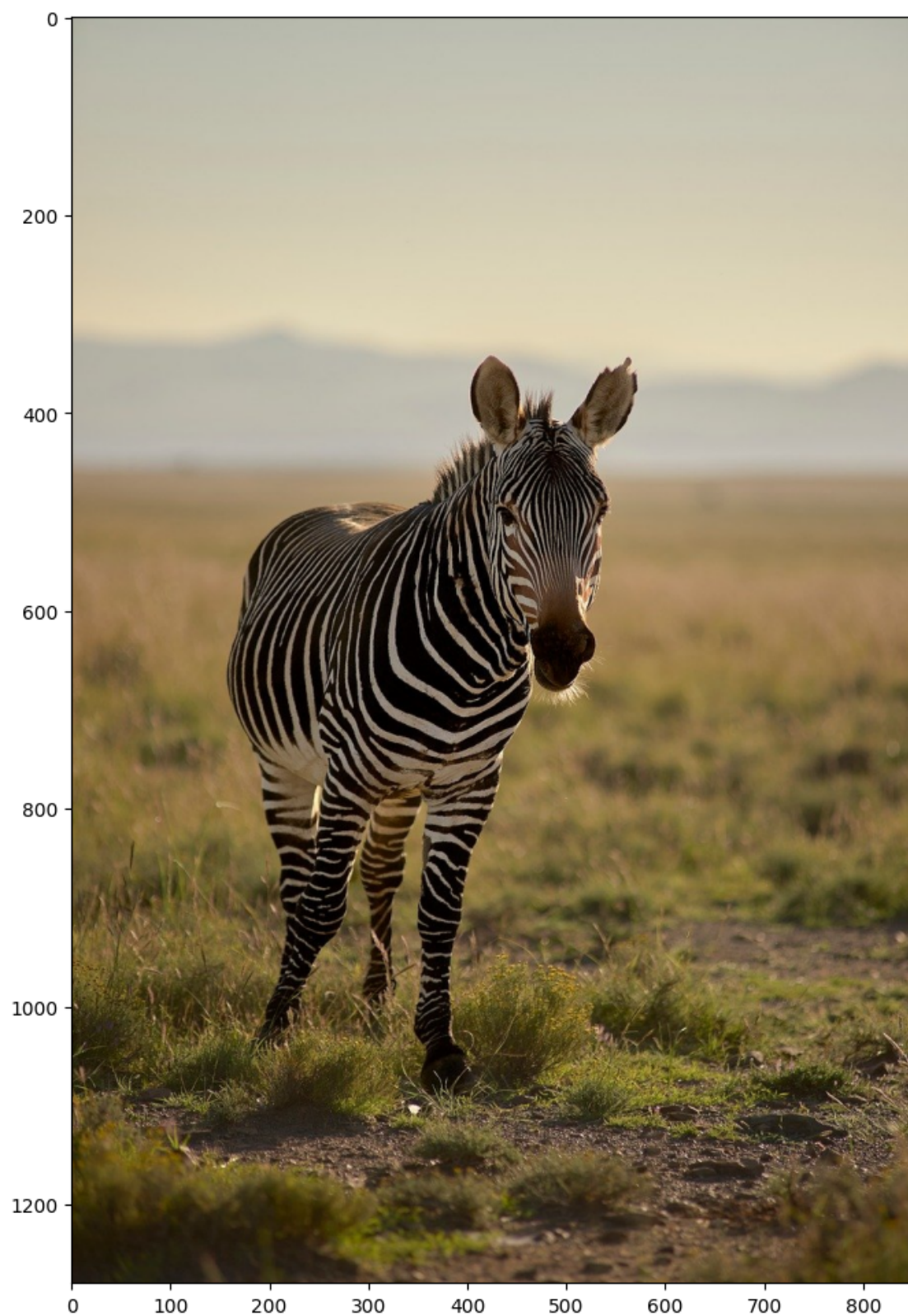
Z analizy wyników wynika, że zastosowania jądra uśredniającego w celu pomniejszania obrazu pozwala zachować detale i ogólny wygląd obrazu choć widać pewne rozmycie. Subiektywne nasostrzejsze wyniki uzyskano przy użyciu jądra  $h_2$  i  $h_1$ . Jądra  $h_3$  i szczególnie  $h_4$  generują największe rozmycie.

Obiektywnie, na podstawie wartości kryterium MSE żadne z jąder nie nadaje się do powiększania obrazów.

Dodatkowo, zastosowanie *max pooling* przy pomniejszaniu i *nearest neighbor* przy powiększaniu daje lepsze wyniki wizualne przez co obrazy są ostrzejsze, kształty bardziej wyraźne, a detale lepiej zachowane. Wartość MSE wskazuje, że ta metoda generuje mniejsze odchylenia od oryginału niż powiększanie przy użyciu kerneli  $h_1$ – $h_4$ , co potwierdza jej większą skuteczność w zachowaniu jakości obrazu.

## 3.2 Skalowanie obrazów kolorowych

### 3.2.1 Porównanie skalowania sekwencyjnego



Rysunek 18: Oryginalne zdjęcie kolorowe

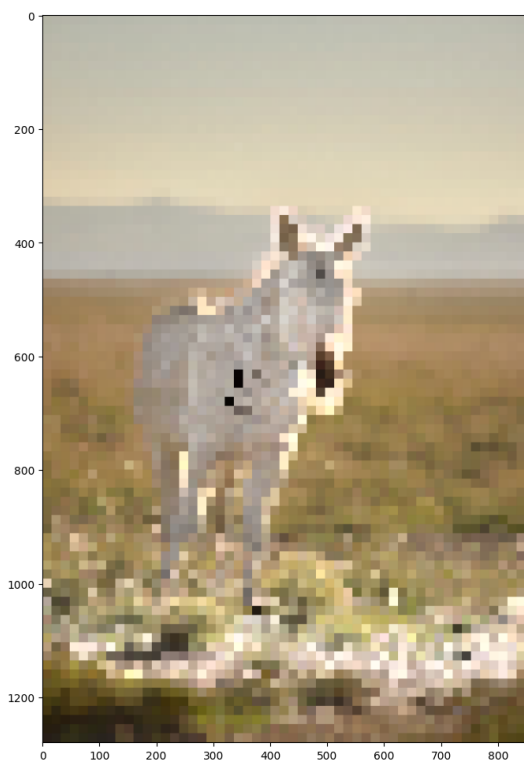


(a) Sekwencyjne powiększanie i pomniejszanie obrazu (2 razy po  $\times 0.5$  i  $\times 2$ )

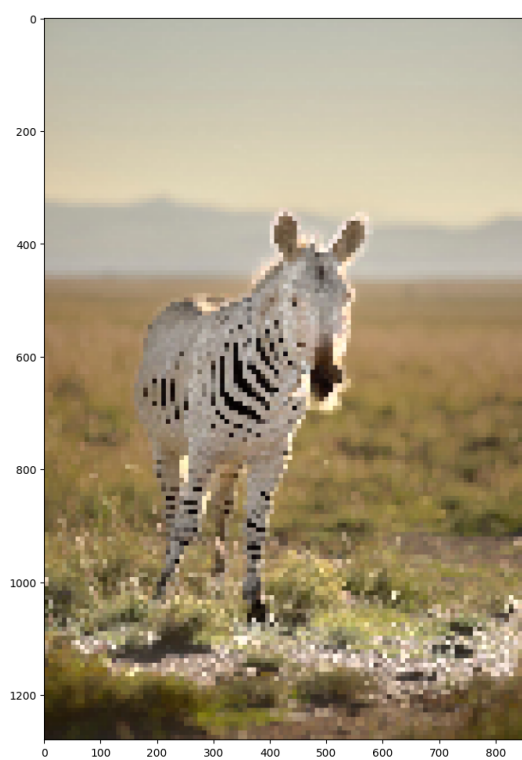


(b) Jednoczesne powiększanie i pomniejszanie obrazu (pomniejszenie o  $\times 0.25$  i powiększenie o  $\times 4$ )

Rysunek 19: Porównanie sekwencyjnego i jednoczesnego skalowania obrazów



(a) Sekwencyjne powiększanie i pomniejszanie obrazu (4 razy po  $\times 0.5$  i  $\times 2$ )



(b) Jednoczesne powiększanie i pomniejszanie obrazu (pomniejszenie o  $\times 0.125$  i powiększenie o  $\times 8$ )

Rysunek 20: Porównanie sekwencyjnego i jednoczesnego skalowania obrazów

|               | Sekwencyjne | Jednoczesne |
|---------------|-------------|-------------|
| <b>2 razy</b> | 0.01218495  | 0.012184953 |
| <b>4 razy</b> | 0.04522232  | 0.026263982 |

### 3.2.2 Wnioski

Analizując obrazu po skalowaniu, widać, że sekwencyjne powiększanie i pomniejszanie prowadzi do większego rozmycia i utraty szczegółów, szczególnie przy większej ilości powtórzeń. Jednoczesne skalowanie, czyli bezpośrednia zmiana rozmiaru zachowuje lepszą ostrość i detale.

Analiza wartości MSE pokazuje, że różnice między metodami są minimalne. Przy czterokrotnym skalowaniu wynik MSE są znacznie wyższe niż dla metody jednoczesnej co potwierdza większe odchylenie obrazu od oryginalnego.